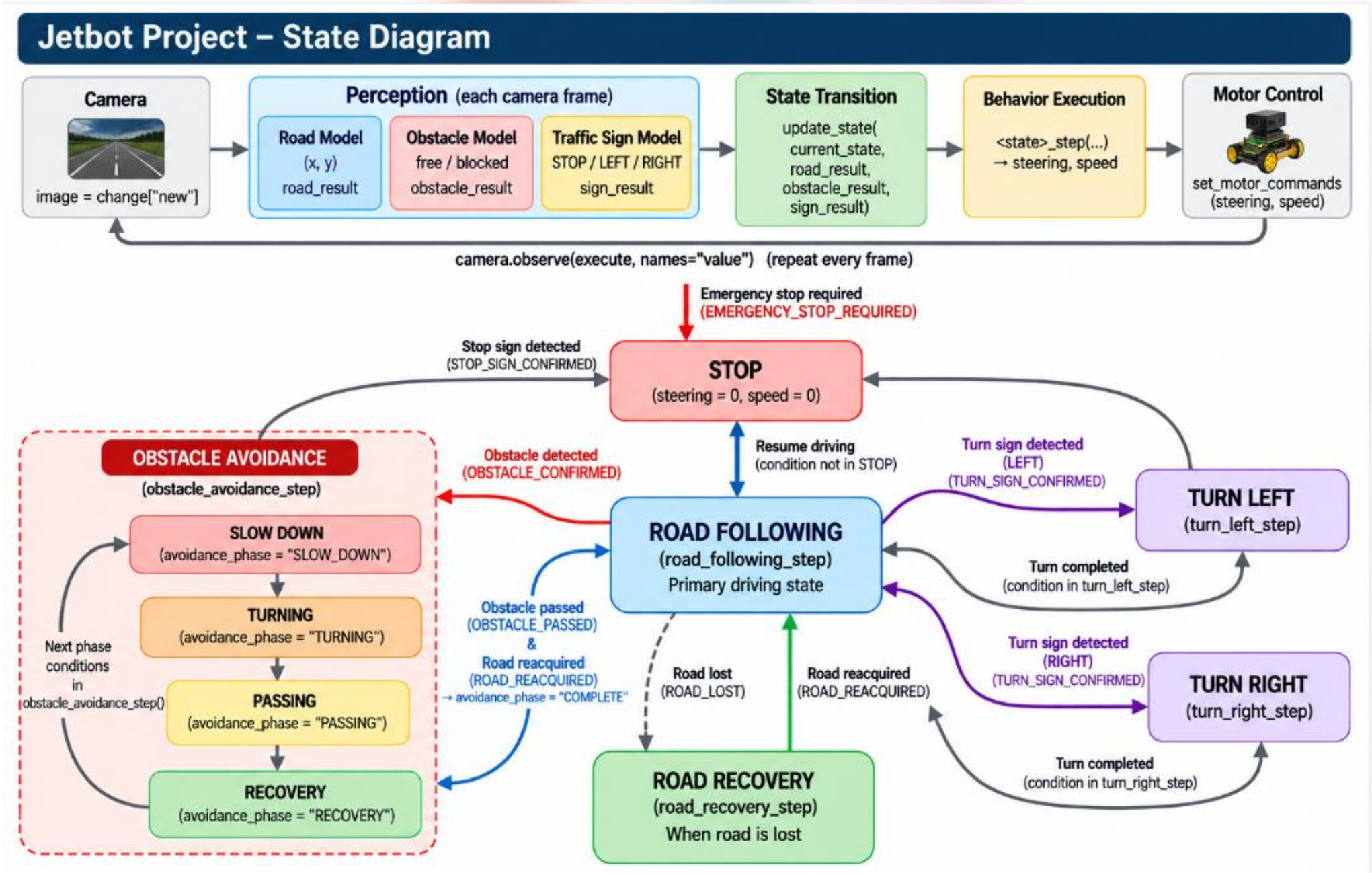
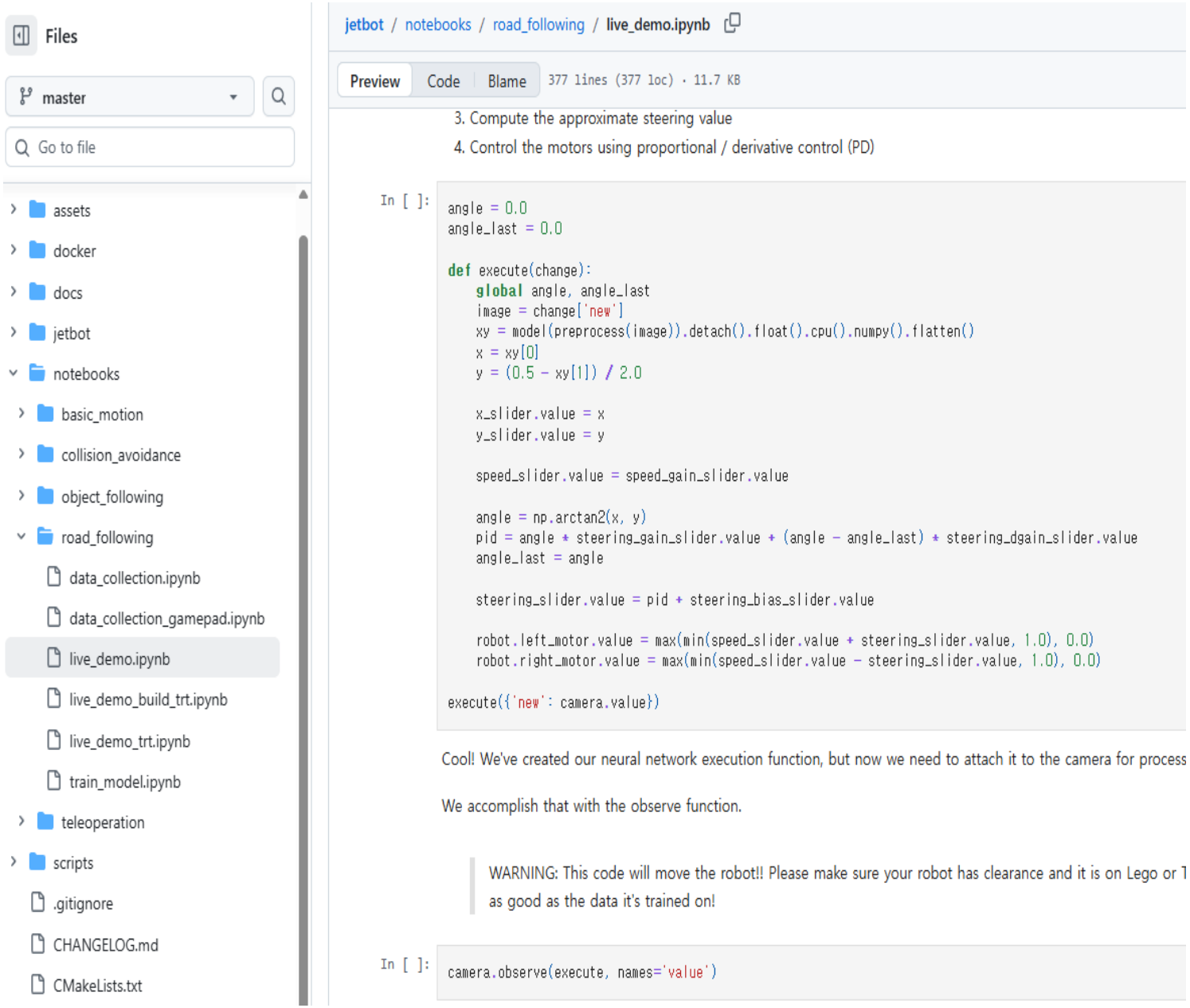
Jetbot Project – Guide #2

State Diagram (Example)



Jetbot Project – Guide #2



Pseudo Code (Example)

```
current_state = "ROAD_FOLLOWING"

def execute(change):
   global current_state

   image = change["new"]

   # 1. Perception
   road_result = road_model(image)
   obstacle_result = obstacle_model(image)
   sign_result = traffic_sign_model(image)

   # 2. State transition
   current_state = update_state(
      current_state,
      road_result,
      obstacle_result,
      sign_result
   )
   # 3. Execute only the selected behavior
   if current_state == "ROAD_FOLLOWING":
      steering, speed = road_following_step(
         road_result
      )
   elif current_state == "OBSTACLE_AVOIDANCE":
      steering, speed = obstacle_avoidance_step(
         obstacle_result, road_result
      )

   elif current_state == “TRAFFIC_SIGN":  {
      …  }
   elif current_state == "STOP":
      steering, speed = 0.0, 0.0

   # 4. Centralized motor control
   set_motor_commands(steering, speed)
```

camera.observe(execute, names="value")

Pseudo Code (Example)

```
FUNCTION update_state(
    current_state,
    road_result,
    obstacle_result,
    sign_result
):

    # 1. Check highest-priority conditions
    IF EMERGENCY_STOP_REQUIRED:
        next_state = "STOP"

    # 2. Handle obstacle detection
    ELSE IF OBSTACLE_CONFIRMED:
        next_state = "OBSTACLE_AVOIDANCE"

    # 3. Handle traffic signs
    ELSE IF STOP_SIGN_CONFIRMED:
        next_state = "STOP"

    ELSE IF TURN_SIGN_CONFIRMED:
        next_state = SELECT_TURN_STATE(
            sign_result
        )

    # 4. Handle road loss
    ELSE IF ROAD_LOST:
        next_state = "ROAD_RECOVERY"

    # 5. Continue normal driving
    ELSE:
        next_state = "ROAD_FOLLOWING"

    RETURN next_state
```

```
FUNCTION obstacle_model(image):

    # 1. Preprocess camera image
    input = PREPROCESS(image)

    # 2. Run obstacle classification model
    prediction = MODEL_INFERENCE(input)

    # 3. Interpret prediction
    free_score = prediction["free"]
    blocked_score = prediction["blocked"]

    # 4. Return perception result
    result.free_score = free_score
    result.blocked_score = blocked_score
    result.is_blocked = CLASSIFY(
        free_score,
        blocked_score
    )

    RETURN result
```

```
FUNCTION traffic_sign_model(image):

    # 1. Preprocess image
    input = PREPROCESS(image)

    # 2. Detect or classify traffic signs
    detections = MODEL_INFERENCE(input)

    # 3. Select relevant sign detections
    relevant_signs = FILTER_DETECTIONS(
        detections
    )

    # 4. Convert detections into semantic information
    result.sign_type = IDENTIFY_SIGN(
        relevant_signs
    )
    result.confidence = GET_CONFIDENCE(
        relevant_signs
    )

    # 5. Return result to the decision system
    RETURN result
```

```
FUNCTION road_following_step(road_result):

    # 1. Read predicted target coordinates
    x = road_result.x
    y = road_result.y

    # 2. Compute heading error
    angle = COMPUTE_ANGLE(x, y)

    # 3. Compute steering command
    steering = COMPUTE_PD_CONTROL(
        angle,
        previous_angle
    )

    # 4. Select driving speed
    speed = BASE_SPEED

    # 5. Save current angle for next control step
    previous_angle = angle

    RETURN steering, speed
```

Pseudo Code (Example)

```
FUNCTION obstacle_avoidance_step(
    obstacle_result,
    road_result
):

    # 1. Initialize or continue avoidance phase
    IF avoidance_phase == "SLOW_DOWN":
        speed = REDUCED_SPEED
        steering = 0

        IF READY_TO_TURN:
            avoidance_phase = "TURNING"

    # 2. Turn away from the obstacle
    ELSE IF avoidance_phase == "TURNING":
        steering = SELECT_AVOIDANCE_STEERING()
        speed = TURNING_SPEED

        IF TURN_COMPLETED:
            avoidance_phase = "PASSING"

    # 3. Move past the obstacle
    ELSE IF avoidance_phase == "PASSING":
        steering = PASSING_STEERING()
        speed = PASSING_SPEED

        IF OBSTACLE_PASSED:
            avoidance_phase = "RECOVERY"

    # 4. Return to the road
    ELSE IF avoidance_phase == "RECOVERY":
        steering = road_recovery_step(road_result)
        speed = RECOVERY_SPEED

        IF ROAD_REACQUIRED:
            avoidance_phase = "COMPLETE"

    RETURN steering, speed, avoidance_phase
```

```
FUNCTION road_recovery_step(road_result):

    # 1. Check whether road information is reliable
    IF ROAD_DETECTED_RELIABLY(road_result):
        steering = COMPUTE_RECOVERY_STEERING(
            road_result
        )
        speed = LOW_SPEED

    # 2. Road is still missing
    ELSE:
        steering = SEARCH_STEERING()
        speed = SEARCH_SPEED

    # 3. Return control command
    RETURN steering, speed
```

```
FUNCTION set_motor_commands(steering, speed):

    # 1. Limit command values
    steering = CLAMP(steering, MIN_STEERING,
                     MAX_STEERING)
    speed = CLAMP(speed, MIN_SPEED, MAX_SPEED)

    # 2. Convert steering and speed to wheel commands
    left_command = speed + steering
    right_command = speed - steering

    # 3. Limit wheel commands
    left_command = CLAMP(left_command, 0, 1)
    right_command = CLAMP(right_command, 0, 1)

    # 4. Apply commands to the robot
    robot.left_motor.value = left_command
    robot.right_motor.value = right_command
```